Studio Nx

JS 286 — 패턴 만들기와 롤 노트 저장

2026-10-19

JavaScript 286 — 패턴 만들기와 롤 노트 저장 (`sequencer.js`)

항목	내용
파일	`studionx/frontend/static/js/pages/sequencer.js`
작업	S1-10a 클립·연결 저장: 패턴 생성/미사용 패턴 정리/롤 노트 저장 및 롤→클립 미리보기 동기화 (FR-2196 클립·패턴, FR-2244 패턴 메뉴)
작성	Claude (AI), 2026-10-19, 사용자 검토
이전 · 다음	이전 285 클립 저장 이어 주기, 다음 287 서버 클립 노트 불러오기

1. 하는 일

패턴을 만들고, 롤에서 고친 노트를 서버 패턴에 저장하며, 롤 변경이 편곡 클립 미리보기에 바로 반영되게 합니다. `saveNewPattern` 은 PATTERNS 메뉴의 `+ 새 패턴` 버튼/명령에서 쓰이고, `initPatternCommands` 는 `pattern.new`/`pattern.purge` 명령을 등록합니다. `purgeUnusedPatterns` 는 클립이 참조하지 않는 패턴만 삭제합니다. `initClipPreviews` 는 `sq:roll-notes` 이벤트를 듣고 `clip.__notes` 를 갱신한 뒤 `saveClipNotes` 로 `PUT /patterns/{id}/notes` 를 지연 전송합니다.

```
PATTERNS 메뉴: + 새 패턴 클릭 / 커맨드 팔레트 'pattern.new'
        ↓
initPatternCommands → button.click() 또는 saveNewPattern
        ↓
POST /patterns { project_id, name, length_tick: 4 bars }
        ↓
patternsById 업데이트 · markDirty('pattern')

MIDI 롤에서 노트 추가/이동/길이변경/삭제
        ↓
sequencer-roll.js emit 'sq:roll-notes' { clip, notes }
        ↓
sequencer.js initClipPreviews → setClipNotes(clip, notes) + clip.__notes = notes + saveClipNotes
        ↓
WeakMap 타이머 400ms 후 PUT /patterns/{patternId}/notes (모든 변경을 모아서 보냄)
```

2. 언제 쓰나

쓰는 곳	
`sequencer-ui.js`/메뉴	`pattern.new` 명령으로 새 패턴을 만듭니다.
`sequencer-commands.js` 커맨드 팔레트	`pattern.new`, `pattern.purge` 명령으로 호출됩니다.
`sequencer-roll.js`	노트가 바뀔 때마다 `sq:roll-notes` 를 발행합니다.
편곡 화면	`initClipPreviews` 가 `setClipNotes` 를 불러 MIDI 클립 미리보기를 다시 그립니다.

3. 코드

3.1 saveNewPattern

```
/**
 * '+ 새 패턴' 을 서버에 만들기 (FR-2196) — 목록 항목에 번호를 달아 두어 그 패턴을 끌어 놓으면
 * 클립이 이 패턴을 가리킵니다.
 * @param {{name: string, item?: HTMLElement}} change
 */
async function saveNewPattern(change) {
  if (!projectIdOf()) return;
  const name = (change.name || 'Pattern').slice(0, 100);
  try {
    const pattern = await api.post('/patterns', {
      project_id: Number(projectIdOf()),
      name: name,
      length_tick: TICKS_PER_BAR * 4,
    });
    patternsById[pattern.pattern_id] = pattern;
    if (globalThis.console) globalThis.console.debug('[sequencer] pattern', pattern.pattern_id);
    markDirty('pattern');
  } catch (err) { clipWarn(err); }
}
```

3.2 initPatternCommands

```
/** 패턴 명령 등록 — PATTERNS 메뉴에서 패턴을 만들거나 안 쓰는 패턴을 지웁니다 (FR-2244) */
function initPatternCommands() {
  registerCommand('pattern.new', function () {
    const btn = document.getElementById('pattern-new');
    if (btn) btn.click();
  }, { label: 'New pattern', group: 'patterns' });
  registerCommand('pattern.purge', purgeUnusedPatterns, { label: 'Remove unused patterns', group: 'patte
rns' });
}
```

- `pattern.new`: `#pattern-new` 버튼을 클릭해 기존 UI 흐름을 타게 합니다.
- `pattern.purge`: `purgeUnusedPatterns` 를 실행합니다.

3.3 purgeUnusedPatterns

```
/** 안 쓰는 패턴 지우기 (FR-2196) — 클립이 가리키지 않는 패턴만 서버에서 지웁니다 (쓰는 패턴은 400) */
async function purgeUnusedPatterns() {
  if (!projectIdOf()) { showToast('Open a project first'); return; }
  const ids = Object.keys(patternsById);
  let removed = 0;
  for (let i = 0; i < ids.length; i += 1) {
    try {
      await api.delete('/patterns/' + encodeURIComponent(ids[i]));
      delete patternsById[ids[i]];
      removed += 1;
    } catch (err) {
      if (!(err instanceof ApiError) || err.status !== 400) clipWarn(err);
```

```
    }
  }
  if (removed) {
    applyPatterns(Object.keys(patternsById).map(function (id) { return patternsById[id]; }));
    markDirty('pattern');
    showToast(removed + ' unused pattern' + (removed > 1 ? 's' : '') + ' removed');
  } else {
    showToast('No unused patterns to remove');
  }
}
```

- 쓰는 패턴을 지우려 하면 서버가 400을 돌려줍니다. 이 경우는 경고만 무시하고 다음 패턴으로 넘어갑니다.
- 지워진 패턴은 로컬 `patternsById` 에서도 뺍니다.
- 목록은 `applyPatterns` 로 다시 그립니다.

3.4 롤 노트 저장과 미리보기 동기화

```
/**
 * 피아노롤 노트 변화 → 그 MIDI 클립 미리보기 다시 그리기 (요구사항 1, 2026-10-17)
 * - sequencer-roll.js 가 노트를 바꿀 때마다 'sq:roll-notes' 를 보낸다
 */
function initClipPreviews() {
  document.addEventListener('sq:roll-notes', (e) => {
    const detail = e.detail || {};
    if (!detail.clip) return;
    setClipNotes(detail.clip, detail.notes || []);
    detail.clip.__notes = detail.notes || [];   // 다시 열 때 채울 노트도 갱신
    saveClipNotes(detail.clip, detail.notes || []);
  });
}
initClipPreviews();
```

```
/** MIDI 클립의 노트를 서버 패턴에 저장 (S1-10a FR-2196) — 피아노롤에서 고칠 때 호출 (조금 미룸) */
const noteSaveTimers = new WeakMap();
function saveClipNotes(clip, notes) {
  const patternId = clip.dataset.patternId;
  if (!patternId || !projectIdOf()) return;
  const prev = noteSaveTimers.get(clip);
  if (prev) window.clearTimeout(prev);
  noteSaveTimers.set(clip, window.setTimeout(function () {
    noteSaveTimers.delete(clip);
    api.put('/patterns/' + encodeURIComponent(patternId) + '/notes', {
      notes: notes.map(function (n) {
        return { pitch: n.pitch, start_tick: n.start_tick, length_tick: n.length_tick, velocity: n.veloc
ity || 100 };
      }),
    }).then(function () { markDirty('notes'); }).catch(clipWarn);
  }, 400));
}
```

- `__notes` 에 현재 노트 목록을 저장해, 피아노롤을 다시 열거나 `/full` 후 펼침 칸을 채울 때 재사용합니다.
- `WeakMap` 으로 클립마다 타이머를 관리해, 롤에서 연달아 고치는 동안엔 마지막 변경 400ms 후 한 번만 `PUT /patterns/{id}/notes` 합니다.

- 저장된 노트는 `markDirty('notes')` 로 저장 대기열에 표시됩니다.
- `setClipNotes` 는 `sequencer.js` 에서 `drawClipPreview` 로 편곡 미리보기를 다시 그립니다.

4. 왜 이렇게

선택	이유
`pattern.new` 명령이 `#pattern-new` 버튼을 클릭	UI 와 커맨드 팔레트를 하나의 경로로 묶어, 버튼을 눌러도/단축키·팔레트로 불러도 같은 동작을 보장합니다.
`purgeUnusedPatterns` 가 400 만 무시	클립이 여전히 가리키는 패턴은 삭제할 수 없습니다. 서버의 제약을 그대로 따르면서, 에러 토스트를 띄우지 않고 다음 패턴을 시도합니다.
롤 노트 저장을 400ms 디바운스	노트를 끌거나 길이를 바꾸는 동안 수십 번 이벤트가 올 수 있습니다. 타이핑/드래그 중엔 모아 두었다가 마지막에 한 번만 `PUT` 으로 요청 수를 줄입니다.
`clip.__notes` 를 갱신	282에서 `clipElement` 가 `patternsById` 의 노트를 `el.__notes` 에 담고, 롤도 이 값을 읽어 채웁니다. 롤에서 고친 뒤에도 이 값을 최신화해 다시 열 때와 미리보기가 일치하게 합니다.
`saveClipNotes` 가 `patternId` 없으면 조용히 리턴	데모 클립이나 패턴 없는 미완성 클립일 수 있으므로, 불필요한 호출을 피합니다.

5. 시험

`docs/handoff/s1-10a_tests/ui.mjs` 관련 항목을 확인합니다.

- PATTERNS 목록에서 `+ 새 패턴` 을 누르면 `POST /patterns` 가 호출되고 새 패턴이 목록에 추가됩니다.
- 커맨드 팔레트에서 `pattern.new` 을 실행하면 같은 동작이 일어납니다.
- 안 쓰는 패턴 정리(`pattern.purge`) 시, 클립이 쓰는 패턴은 건너뛰고 안 쓰는 패턴만 삭제됩니다.
- 피아노롤에서 노트를 추가/이동/삭제하면 `sq:roll-notes` 가 발행되고, 편곡의 MIDI 클립 미리보기에 즉시 반영됩니다.
- 롤에서 노트를 고친 뒤 400ms 이내에 다시 고치면 `PUT /patterns/{id}/notes` 는 마지막 한 번만 전송됩니다.
- 롤을 열고 닫은 뒤 다시 열면 마지막으로 저장된 노트가 그대로 불러와집니다.
- 저장 성공 시 `markDirty('notes')` 가 실행됩니다.